

Reviewer Pack — Minimal Local Induction Dimension and Boolean Circuit Monoto...

Entry 35cbb117bac8 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 21:02:02 UTC

Minimal Local Induction Dimension and Boolean Circuit Monotone Width

Entry ID: 35cbb117bac8 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 21:02:02 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology (Local Induction Dimension) **Field B** (complexity object): Boolean Circuit Complexity (Circuit Monotone Width)

Statement:

The minimal local induction dimension (lnd) of a Boolean function is linearly correlated with its circuit monotone width, such that $\text{lnd}(\varphi) = \Theta(w_M(\varphi))$, where $w_M(\varphi)$ denotes the monotone width of the Boolean function φ .

Rationale (proposer's reasoning):

Local Induction Dimension (lnd) measures the complexity of a space by counting the minimum number of simplices needed to approximate it. In algebraic topology, lnd is used to study the structure of spaces. By connecting this concept with circuit monotone width, which measures the complexity of Boolean functions, we might uncover new insights into the interplay between geometric and computational complexities.

Taxonomy category: `local_induction_dimension_circuit_monotone_width` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f3ba679ccfe0a804

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient between $\text{lnd}(\varphi)$ and $w_M(\varphi)$ across all 30 seeds is ≥ 0.8 , with no seed producing a correlation coefficient < 0.5 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): -lnd AND Boolean Circuit Complexity - monotone width IN BOOLEAN LOGIC AND Local Induction Dimension - circuit monotone width AND local induction dimension

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Generate a random Boolean function with n variables
    n = 10
     $\phi$  = [random.choice([0, 1]) for _ in range(2**n)]

    # Calculate the minimal local induction dimension (lnd( $\phi$ ))
    # This is a placeholder implementation. Replace with actual calculation.
    lnd_value = random.uniform(n // 2, n)

    # Calculate the monotone width of  $\phi$ 
    w_M_value = random.randint(n // 4, n // 2)

    return {
        "metric_name": "correlation",
        "metric_value": lnd_value * w_M_value,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(2, 1000) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
```

```

results.append(result)

mean_value = sum(r["metric_value"] for r in results) / len(results)
std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] and r["metric_value"] < 0.5 for r in results):
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"] and r["metric_value"] < 0.5)
    print(f"RESULT: FALSIFIED counterexample=\"low_correlation\" first_failing_seed={first_failing_seed}")
else:
    print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

xample': 'mapping_undefined'}
TRIAL: {'metric_name': 'correlation', 'metric_value': 23.5483198020628, 'instances_tested': 1, 'n_m
TRIAL: {'metric_name': 'correlation', 'metric_value': 32.485324251119245, 'instances_tested': 1, 'n
TRIAL: {'metric_name': 'correlation', 'metric_value': 29.474944483488613, 'instances_tested': 1, 'n
TRIAL: {'metric_name': 'correlation', 'metric_value': 28.77113982340864, 'instances_tested': 1, 'n_m
TRIAL: {'metric_name': 'correlation', 'metric_value': 37.0896134105054, 'instances_tested': 1, 'n_ma
TRIAL: {'metric_name': 'correlation', 'metric_value': 12.377670884161155, 'instances_tested': 1, 'n
TRIAL: {'metric_name': 'correlation', 'metric_value': 10.441949751579862, 'instances_tested': 1, 'n
TRIAL: {'metric_name': 'correlation', 'metric_value': 16.342942244124256, 'instances_tested': 1, 'n
TRIAL: {'metric_name': 'correlation', 'metric_value': 35.24176707375404, 'instances_tested': 1, 'n_m
TRIAL: {'metric_name': 'correlation', 'metric_value': 20.45108179563455, 'instances_tested': 1, 'n_m
TRIAL: {'metric_name': 'correlation', 'metric_value': 25.10662247470828, 'instances_tested': 1, 'n_m
RESULT: INCONCLUSIVE support_fraction=0.0

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test code uses random values for both the minimal local induction dimension (lnd) and the monotone width, which are not calculated based on any mathematical definition of these concepts. This makes it impossible to confirm or falsify the conjecture with this test.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code uses random values for both the minimal local induction dimension (lnd) and the monotone width, which are not calculated based on any mathematical definition of these concepts. This makes it impossible to confirm or falsify the conjecture with this test. | next: Develop a proper mathematical framework for calculating lnd and monotone width, then retest the conjecture using this framework.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16484
2	propose	ollama_remote	glm4:latest	0	0	12658
3	preregistration	ollama_remote	glm4:latest	0	0	9019
4	novelty	ollama_remote	glm4:latest	0	0	8073
5	novelty	ollama_remote	glm4:latest	0	0	8976
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16009
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10905
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10903
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6029
10	critic	ollama_remote	glm4:latest	0	0	28984
11	judge	ollama_remote	glm4:latest	0	0	9355

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 137395 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/35cbb117bac8.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/35cbb117bac8.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/35cbb117bac8.tar.gz` (if generated)